Post-Quantum Migration of the SMB Protocol using a Quantum-Safe Link Layer

Ahmad Abdellatif, George Ghanem, Tarek Kaadan

November 2025

Abstract

This project studies how to migrate a file sharing protocol to post-quantum cryptography. The protocol chosen is SMB (server message block). Instead of modifying the SMB code,a quantum-safe link is implemented in Rust and placed between the client and the Windows server. The link uses NIST-standardised post-quantum algorithms which are ML-KEM for key encapsulation and ML-DSA for digital signatures, and supports a hybrid mode that combines them with classical elliptic-curve primitives. This report describes the protocol design, the migration strategy, and the performance and security impact of adding post-quantum primitives in front of a real-world protocol.

1 Introduction

Large-scale quantum computers threaten public-key cryptography that is used today in most network protocols. Algorithms such as RSA and elliptic curve Diffie-Hellman can be broken by Shor's algorithm, which means that encrypted traffic recorded today may be decrypted in the future (harvest now, decrypt later). For this reason, there is a strong incentive to migrate current systems to post-quantum cryptography (PQC) based on problems that are supposedly hard for both classical and quantum computers to solve.

In this project we focus on SMB, a widely used protocol for file sharing on local networks. SMB is performance-sensitive and already uses modern symmetric cryptography which makes it a good choice for PQ migration. Instead of changing the SMB protocol, we designed and implemented a quantum-safe link layer that runs the PQ key exchange and then forwards SMB traffic over an encrypted channel.

The link layer is implemented in Rust. It uses a TCP listening port on the client side ,then performs a handshake based on ML-KEM and ML-DSA in combination with classical X25519, and after that it derives symmetric session keys using HKDF, and finally acts as a proxy between the SMB client and the SMB server. Our goal is to understand both conceptual and practical aspects of the migration, what changes are required, how much the overhead increases, and how to make the solution hybrid.

2 Protocol Selection and Analysis

2.1 Rationale for Choosing SMB

SMB was selected as the target protocol since it is a core component of Windows networks and is heavily used for file and printer sharing.

2.2 Current Cryptographic Primitives in SMB

In a typical deployment, SMB runs over TCP and uses separate authentication schemes like NTLM or Kerberos. These rely on classical public-key primitives like RSA certificates, elliptic-

curve key exchange and signatures, or classical password-based schemes. Once a session key has been established, SMB3 uses symmetric algorithms such as AES-CCM or AES-GCM to provide confidentiality and integrity for file data and metadata.

The consequence is that the long-term security of SMB sessions depends on classical public-key cryptography. An adversary who uses the "harvest now, decrypt later" strategy could attack the key exchange or the authentication phase and recover the session keys. Protecting against such attacks is one of the motives for making this protocol quantum-safe.

2.3 Security Requirements and Constraints

The migration considered in this project aims to satisfy the following security requirements:

- Confidentiality and integrity: File contents and protocol metadata must remain protected against eavesdropping and tampering.
- Forward secrecy: Compromise of long-term keys in the future should not allow decryption of past SMB sessions.
- Public-key operations used in the handshake should not be vulnerable to known quantum algorithms.

At the same time there are important practical constraints:

- The Windows SMB implementation is treated as a black box and is not modified.
- The added latency and cost of the new PQ link layer should remain small for normal file sharing workloads in a lab environment.
- The solution should coexist with clients that do not support post-quantum cryptography.

3 Post-Quantum Integration Design

3.1 Cryptographic Building Blocks

The quantum-safe link uses a combination of post-quantum and classical primitives. On the PQ side, Kyber-768 is used as a key-encapsulation mechanism and Dilithium2 is used for digital signatures. On the classical side, the implementation uses X25519 as an optional elliptic-curve Diffie-Hellman component that can be enabled in a hybrid mode.

Session keys are derived using HKDF with SHA-256. The keying material is formed by concatenating the Kyber shared secret with the optional X25519 shared secret when hybrid mode is enabled. The HKDF output is split into two 32-byte keys, one for recieving and one for sending, and these keys are used with the ChaCha20-Poly1305 AEAD to protect individual frames of SMB data. Signatures are always produced and verified with Dilithium2.

3.2 Handshake Protocol

The handshake between the client link and the server link consists of three messages: ClientHello, ServerHello and ClientAuth. Each message is framed with a 4-byte magic value to identify the type, a protocol-version field and a length-prefixed payload.

On the client side, the link first constructs a "Capabilities" structure that lists the supported KEM, the supported signature algorithm, a hybrid flag, and a pq_required flag. From this it builds a "ClientHello" containing:

- a 32-byte client random
- an ephemeral X25519 public key
- an ephemeral Kyber-768 public key
- the encoded capabilities

This message is sent as the first handshake.

The server decodes the "Client hello", verifies that Kyber-768 and Dilithium2 appear in the "Capabilities" and checks the pq_required bit. It then generates its own 32-byte random value,

an ephemeral X25519 key pair, and uses the client's Kyber public key to perform encapsulation. This produces a Kyber shared secret and a Kyber ciphertext. If both sides have hybrid mode enabled, the server also computes a classical X25519 shared secret.

Next, the server chooses a single suite (k-768, Dilithium2, and a boolean for the mode) and builds a transcript that includes each party's random value, the X25519 public keys, the Kyber public key and ciphertext, and the encoded capabilities and the suite. This transcript is signed with Dilithium2 using the server's long-term secret key, and the resulting "ServerHello" is sent to the client.

Upon receiving the "ServerHello", the client verifies that the chosen suite uses Kyber-768 and Dilithium2, recomputes the handshake transcript and verifies the Dilithium2 signature with the server's public key. It then decapsulates the Kyber ciphertext using its Kyber secret key and if the server selected the hybrid option, derives the matching X25519 shared secret.

Both parties hash the transcript with SHA-256 to obtain a pre-auth hash which is used as salt to HKDF. The shared secrets are combined to form the HKDF input keying material. HKDF using SHA-256 is then used to derive two 32-byte session keys labelled for the each direction. On the server, these directions are swapped so that both endpoints agree on which key to use in each direction.

Finally, the client produces a "ClientAuth" message that contains a Dilithium2 signature over the same transcript, using the client's long-term secret key. The server verifies this signature against the client's public key. If everything checks out, the handshake is complete and both sides start forwarding SMB traffic over an encrypted channel using ChaCha20-Poly1305 with the session keys.

3.3 Hybrid Mode and Backwards Compatibility

The prototype supports two internal modes for the link:

- PQ-only mode where the session key is derived only from the Kyber-768 shared secret.
- Hybrid mode where an additional X25519 shared secret is computed and combined with the Kyber shared secret before it is passed to HKDF.

The selected mode is encoded in the capabilitiesc and in the chosen suite. In the current implementation the pq_required flag is always set to true on both client and server, so Kyber-768 and Dilithium2 are mandatory and there is no classical-only handshake inside the link layer.

Backwards compatibility is achieved at the network level. The Windows SMB server continues to listen on port 445 for legacy clients, which use the existing classical SMB without any changes. The quantum-safe link listens on a separate TCP port, performs the post quantum handshake described above, and then tunnels SMB traffic to the server. Post-quantum clients connect to this port, while older clients can continue to talk directly to the SMB service in the usual way.

4 Migration Strategy and Negotiation

This section describes how the quantum-safe link fits into a realistic deployment and how it could be rolled out.

4.1 Deployment Topology

The implementation used in this project runs the link on a separate machine and forwards traffic to a server. The idea is that the SMB service continues to listen on its normal port, while the quantum-safe link uses a second port that offers post-quantum protection.

4.2 Negotiation Mechanism

In the current prototype, negotiation is done out-of-band through command-line flags. A more complete design would use a protocol byte that encodes "classical", "PQ" or "hybrid". Both sides would advertise their supported modes and select the strongest common option which would prevent downgrade attacks.

5 Performance Evaluation

5.1 Experimental Setup

All experiments were done in a controlled environment using a Windows 11 laptop as the SMB server and a Kali Linux virtual machine as the client. The Windows machine runs Windows 11 on an Intel Core i5-11400H CPU with 16GB RAM. The SMB service and the server binary of the quantum-safe link both run directly on the Windows host. The client side runs on Kali Linux 2025.2 inside VMware Workstation Pro.

The Windows SMB server listens on the default port 445. The link server listens on port 7445 and forwards traffic to 127.0.0.1:445. On the client side, the link client listens on 127.0.0.1:1445 and connects to the windows IP port 7445. The Kali VM uses smbclient on version SMB3 to access a shared folder on the Windows host.

Three scenarios are compared:

1. Direct SMB : the client connects directly to the Windows SMB server on port 445.
2. PQ-only link: the client connects to the local link on port 1445. the link uses Kyber-768 only to derive the tunnel keys.
3. Hybrid link: The client connect to the local link on port 1445 but the link combines Kyber-768 with X25519 in the HKDF input.

5.2 Handshake Latency

Handshake latency was measured using the time command while running smbclient -L to list the shares. For each scenario the command was executed five times. The measured times in seconds were:

- Direct SMB: 3.93, 2.73, 2.45, 2.73, 2.78
- PQ-only link: 10.34, 4.42, 3.93, 3.72, 3.98
- Hybrid link: 4.32, 3.34, 3.28, 3.41, 3.37

Table 1: Handshake latency for smbclient -L

Scenario	Mean [s]	Median [s]
Direct SMB	2.92	2.73
PQ-only link	5.28	3.98
Hybrid link	3.54	3.37

The PQ-only link almost doubles the average handshake time compared to the direct connection. This is due to one slow first run of the PQ-only link (around 10s), probably caused by cold caches and initial key generation. If the median is considered however, the overhead is smaller. The PQ-only median is about 1.5s slower than the direct median. The hybrid mode sits in between. Its mean and median are only about 0.6s slower than the direct ones.

5.3 File Transfer Throughput

To measure throughput, two test files were created in the Windows share folder: test_10M.bin (10MB) and test_100M.bin (100MB). The client downloaded each file five times using smbclient with a get command wrapped by a time command. The individual real times in seconds were:

Direct SMB

- 10MB: 3.24, 2.56, 2.37, 2.38, 2.24
- 100MB: 3.37, 3.25, 2.85, 2.70, 3.31

PQ-only link

- 10MB: 3.13, 4.37, 2.73, 2.83, 2.98
- 100MB: 3.45, 3.02, 3.05, 2.97, 2.83

Hybrid link

- 10MB: 3.37, 2.36, 2.29, 2.64, 2.43
- 100MB: 2.95, 3.06, 2.95, 3.03, 2.75

From these measurements we compute the average time and the corresponding throughput. The results are shown in Table 2.

Table 2: Average download time and throughput for 10MB and 100MB files.

Scenario	Time 10MB [s]	Thru 10 MB [MB/s]	Time 100MB [s]	Thru 100MB [MB/s]
Direct SMB	2.56	3.9	3.10	32.3
PQ-only link	3.21	3.1	3.06	32.6
Hybrid link	2.62	3.8	2.95	33.9

For the 10MB file, the PQ-only link shows a slight slowdown compared to the direct case (about 3.1MB/s vs 3.9MB/s on average). The hybrid mode is very close to the direct case (around 3.8MB/s). For the 100MB file, all three scenarios have very similar throughput: between 32MB/s and 34MiB/s, with the hybrid mode even slightly ahead of the direct mode.

5.4 Performance Discussion

The experiments show that the main performance cost of the quantum-safe link appears during the handshake. The PQ-only mode doubles the average connection time compared to a direct connection, and the hybrid configuration increases it by about 20 %. When the tunnel is established, the impact on larger file transfers is much smaller. For example, for the 100MB file, the measured throughput of all three scenarios is essentially the same.

6 Security Analysis

Rather than modifying SMB itself, the link creates a secure channel with properties similar to those of modern key-exchange protocols but using post-quantum primitives.

Post-Quantum Key Establishment

The tunnel derives its session keys from Kyber-768, which replaces the classical Diffie–Hellman step normally used. An attacker that uses harvest now, decrypt later cannot recover the shared secret even with a quantum computer.

When hybrid mode is enabled, the Kyber secret is combined with an ephemeral X25519 secret before being passed to HKDF. This provides in-depth security where the tunnel remains secure as long as at least one of the hardness assumptions holds. Hybrid mode also prevents downgrade attacks because the suite is authenticated inside the handshake transcript.

Authentication and Integrity

Long-term Dilithium2 keys are used to authenticate both endpoints. The server signs the handshake transcript in the "ServerHello", and the client authenticates itself in the "ClientAuth". Any modification of public keys, capabilities, or mode selection invalidates the signatures. This prevents man-in-the-middle attacks and ensures that both sides agree on the same keying material and security mode.

After the handshake, all SMB frames are protected using ChaCha20–Poly1305. This provides confidentiality and integrity for the forwarded traffic and ensures that tampering or injection attempts are detected.

Forward Secrecy and Post-Compromise Security

Both Kyber and X25519 keys are ephemeral. Compromise of the long-term keys does not reveal past session keys, and compromise of one session key does not allow to derive the next one. HKDF also produces different keys for each direction of traffic, limiting the impact of any single-key exposure.

Limitations

The link does not secure the SMB server configuration itself, and it doesnt prevent DoS attacks or traffic analysis. SMB authentication remains unchanged and inherits its own security properties. These aspects are outside the scope of the tunnel and must be handled by the underlying system.

Summary

Overall, the quantum-safe link provides confidentiality, integrity, and mutual authentication using post-quantum algorithms while remaining compatible with the original SMB. The design protects SMB sessions against future quantum adversaries with minimal changes to the existing network stack.

7 Conclusion

This project shows that it is possible to add post-quantum protection to the SMB protocol without modifying the server implementation. A separate quantum-safe link layer implemented in Rust can perform a hybrid key exchange using ML-KEM and X25519, derive symmetric session keys, and then tunnel SMB traffic between a client and a server.

References

[1] National Institute of Standards and Technology (NIST), "Post-Quantum Cryptography Standardization," project web page, 2016–ongoing. Available at `https://csrc.nist.gov/projects/post-quantum-cryptography`.

[2] National Institute of Standards and Technology (NIST), *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, U.S. Dept. of Commerce, Aug. 2024.

[3] National Institute of Standards and Technology (NIST), *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*, U.S. Dept. of Commerce, Aug. 2024.

[4] G. Alagic et al., "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency/Internal Report (NIST IR) 8413, 2022.

[5] G. Alagic et al., "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency/Internal Report (NIST IR) 8545, 2025.

[6] T. L. Astrizi, M. Brotman, and A. Aziz, "Seamless Transition to Post-Quantum TLS 1.3: A Hybrid KEMTLS Variant," *Sensors*, vol. 24, no. 22, article 7300, 2024.

[7] B. Kivalo et al., "Post-Quantum Cryptography Recommendations for TLS and SSH," Internet-Draft, IETF UTA Working Group, draft-ietf-uta-pqc-app, work in progress, 2025.